



中山大学 软件工程学院

SUN YAT-SEN UNIVERSITY SCHOOL OF SOFTWARE ENGINEERING

SSE206 《计算机网络》

第三章 传输层

南雨宏

nanyh@mail.sysu.edu.cn

2026-04-01

第 3 章：提纲

3.1 概述和传输层服务

3.2 多路复用与解复用

3.3 无连接传输：UDP

3.4 可靠数据传输的原理

3.5 面向连接的传输：TCP

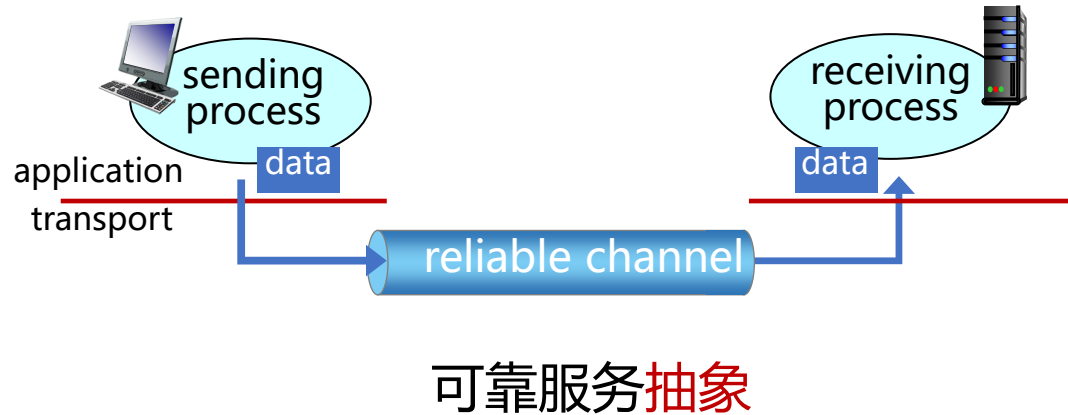
- 段结构
- 可靠数据传输
- 流量控制
- 连接管理

3.6 拥塞控制原理

3.7 TCP拥塞控制

3.8 传输层功能的演化

可靠数据传输 (rdt) 的原理



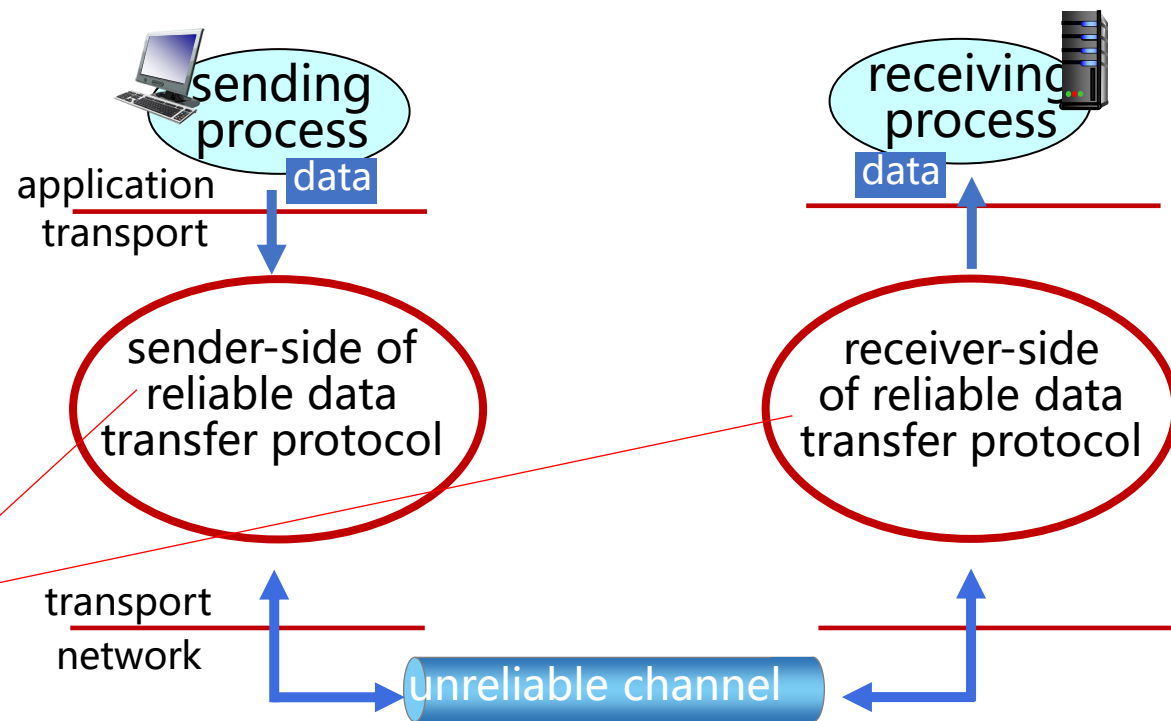
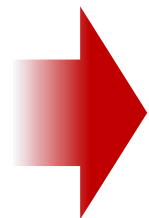
- RDT在应用层、传输层和数据链路层都很重要
- 是网络当中Top-10问题之一

可靠数据传输 (rdt) 的原理



可靠服务抽象

信道的天然的不可靠特性（分组丢失、出错、乱序）决定了可靠数据传输协议（rdt）的复杂性。



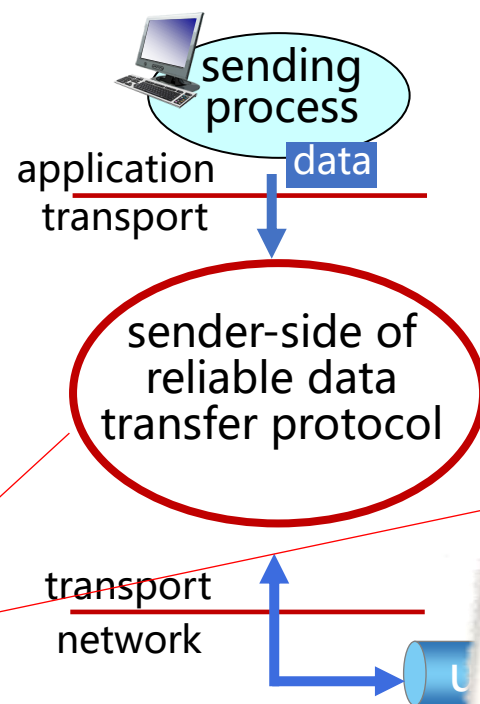
可靠服务实现

可靠数据传输 (rdt) 的原理

- 信道的不可靠特点决定了可靠数据传输协议 (rdt) 的复杂性

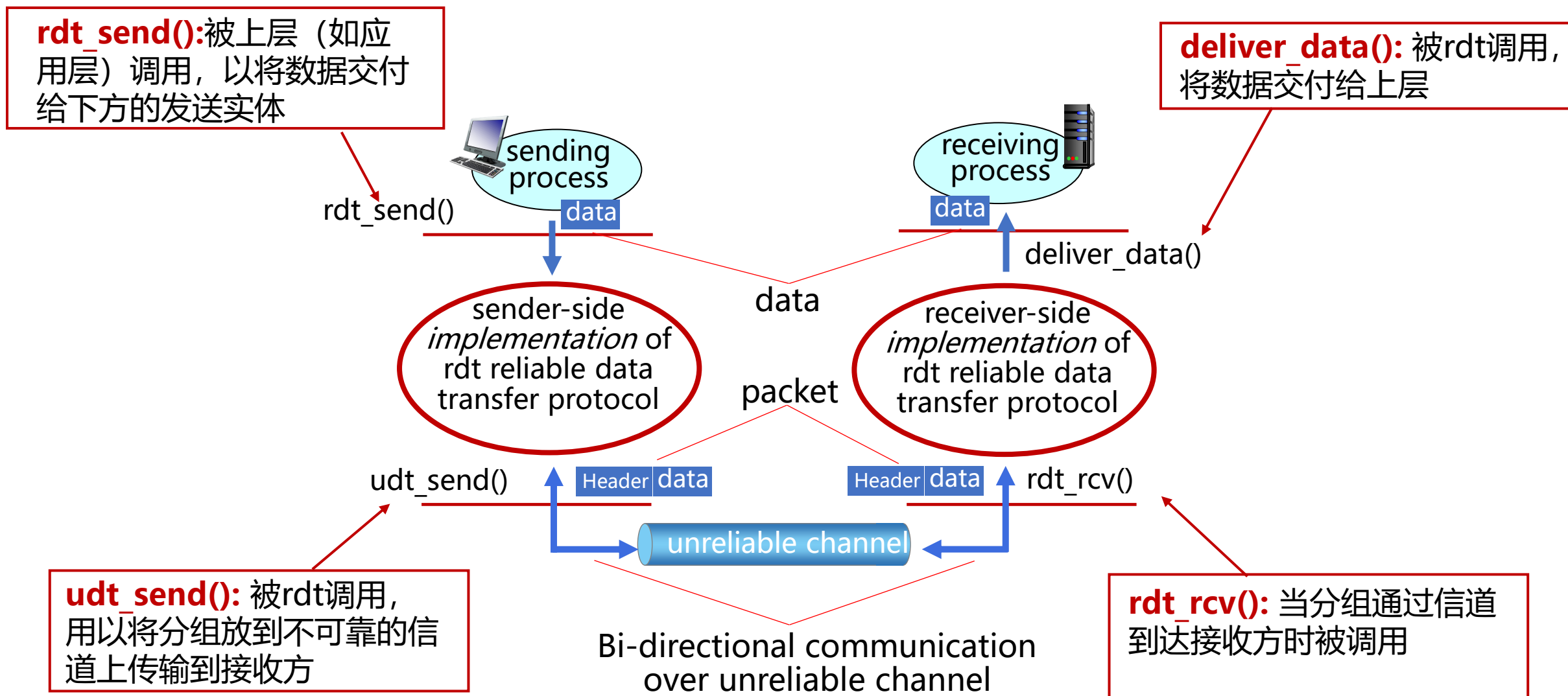
收、发双方并不知道对方的状态 —
— 发出的消息是否收到?

- 除非通过特定消息进行沟通/确认



可靠服务实现

可靠数据传输(rdt): 问题描述

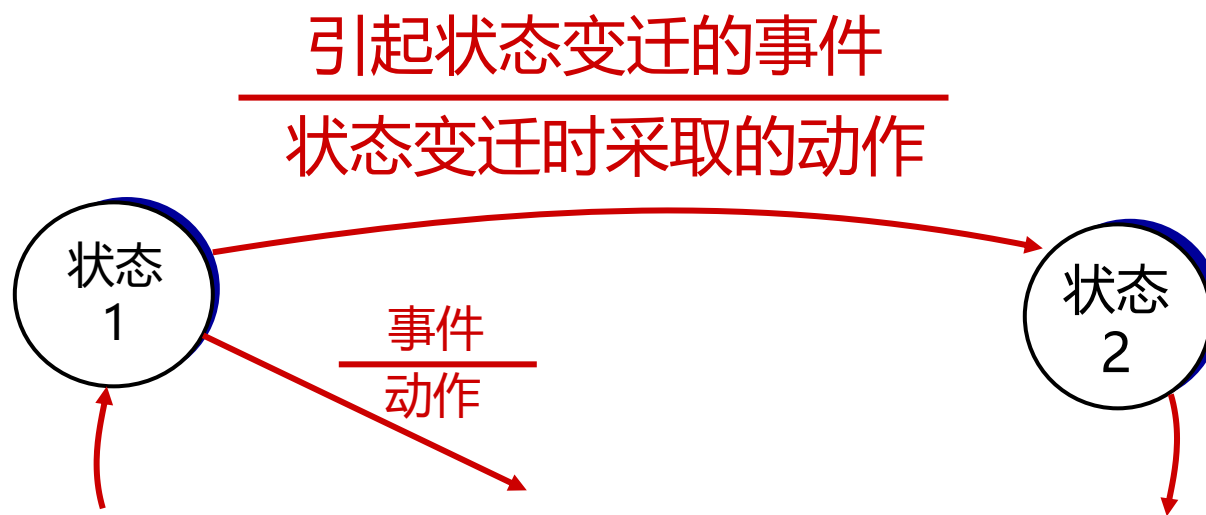


可靠数据传输(rdt): 问题描述

我们将:

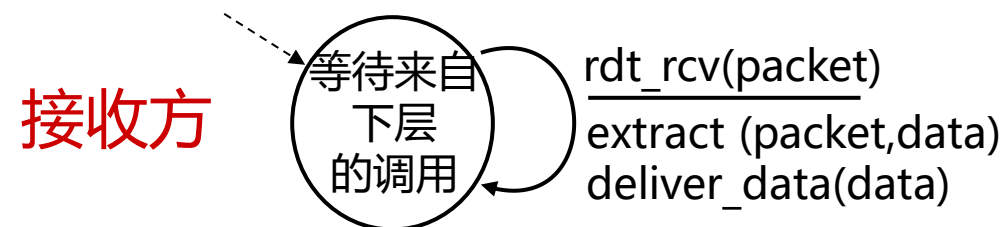
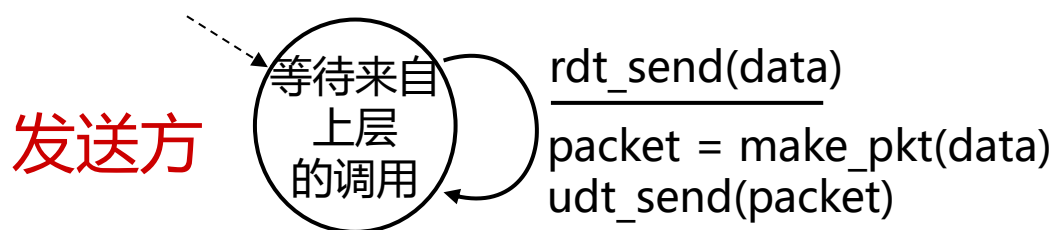
- 渐增式地开发可靠数据传输协议 (rdt) 的发送方和接收方
- 只考虑单向数据传输
 - 但控制信息是双向流动的!
- 双向的数据传输问题实际上是2个单向数据传输问题的综合
- 使用有限状态机(FSM) 来描述发送方和接收方

状态: 在该状态时, 下一个状态只由下一个事件唯一确定



rdt1.0: 可靠信道上的可靠数据传输

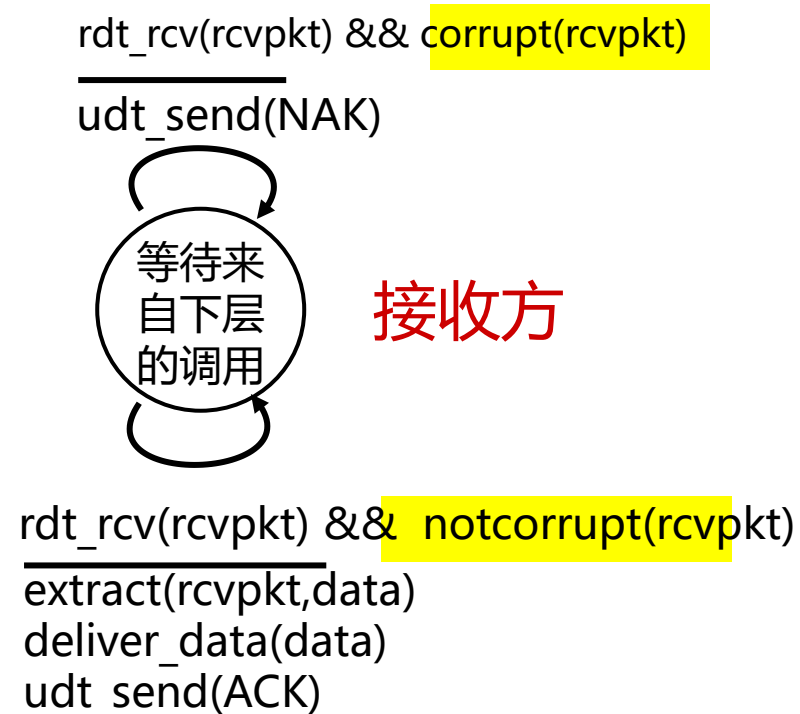
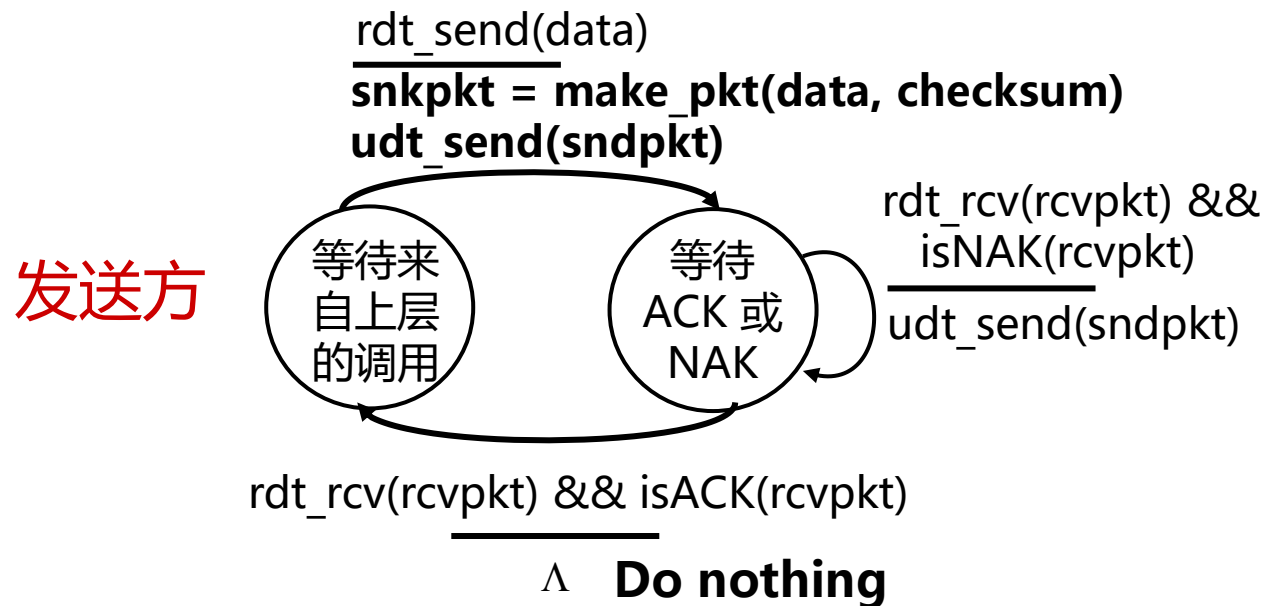
- 下层信道完全可信
 - 没有比特出错
 - 没有丢包
- 发送方和接收方的FSM:
 - 发送方将数据发送到下层信道
 - 接收方将数据从下层信道取出



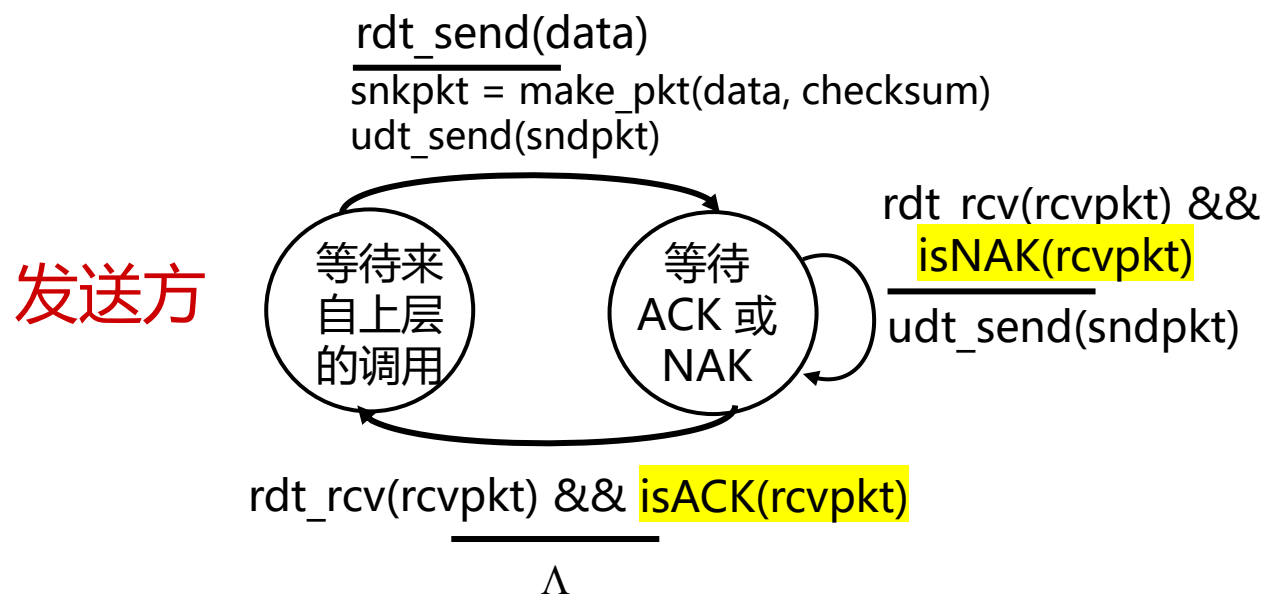
rdt2.0: 具有比特差错的信道

- 下层信道可能会出错：将分组中的比特翻转
 - 使用校验和Checksum来检测比特差错
- 问题：怎样从差错中恢复？
 - 确认(ACK)：接收方显式地告诉发送方分组已被正确接收
 - 否定确认(NAK)：接收方显式地告诉发送方分组发生了差错
 - 发送方收到NAK后，发送方重传分组
- **rdt2.0** 中的新机制：采用差错控制编码进行差错检测
 - 发送方差错控制编码、缓存
 - 接收方使用编码检错
 - 接收方的反馈:控制报文(ACK, NAK):接收方→发送方
 - 发送方收到反馈相应的动作

rdt2.0: FSM 描述



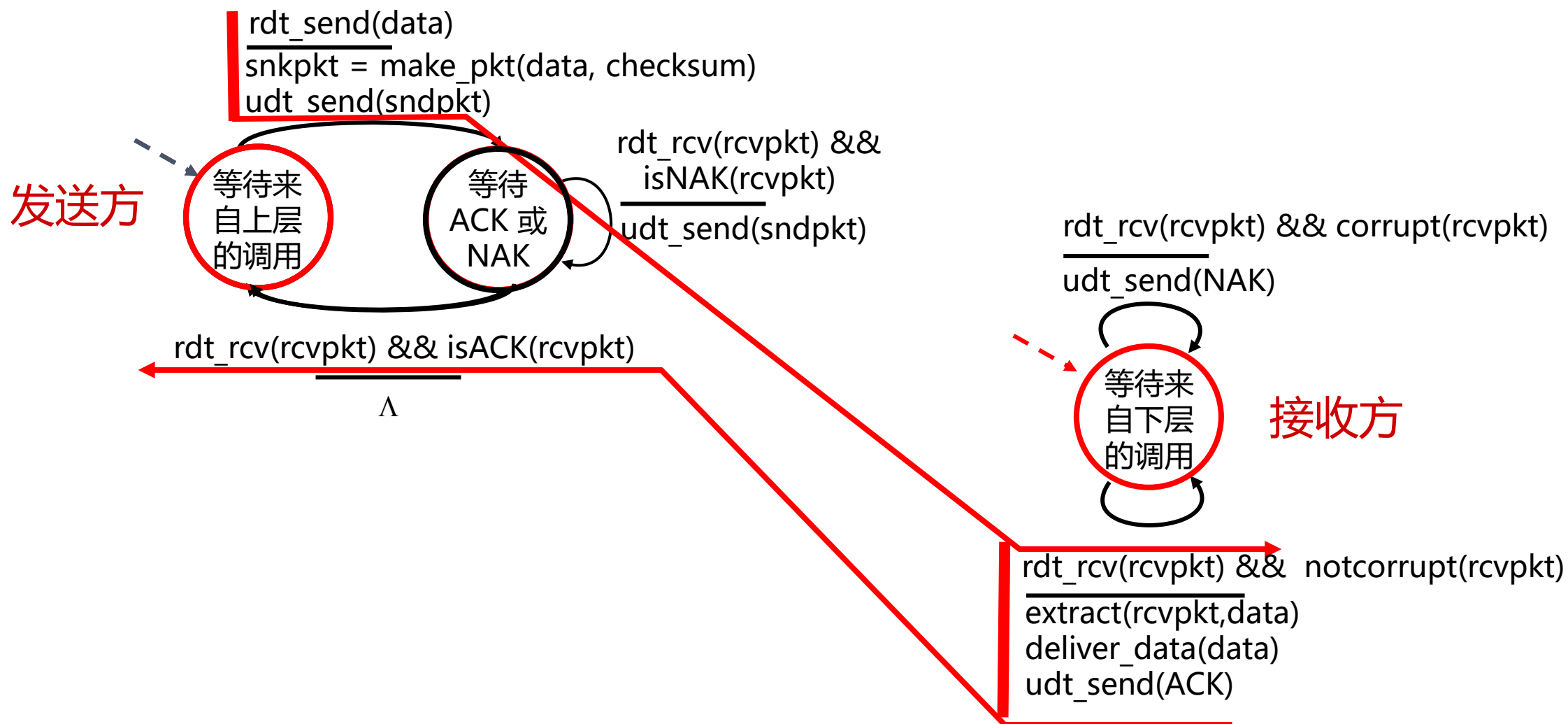
rdt2.0: FSM 描述



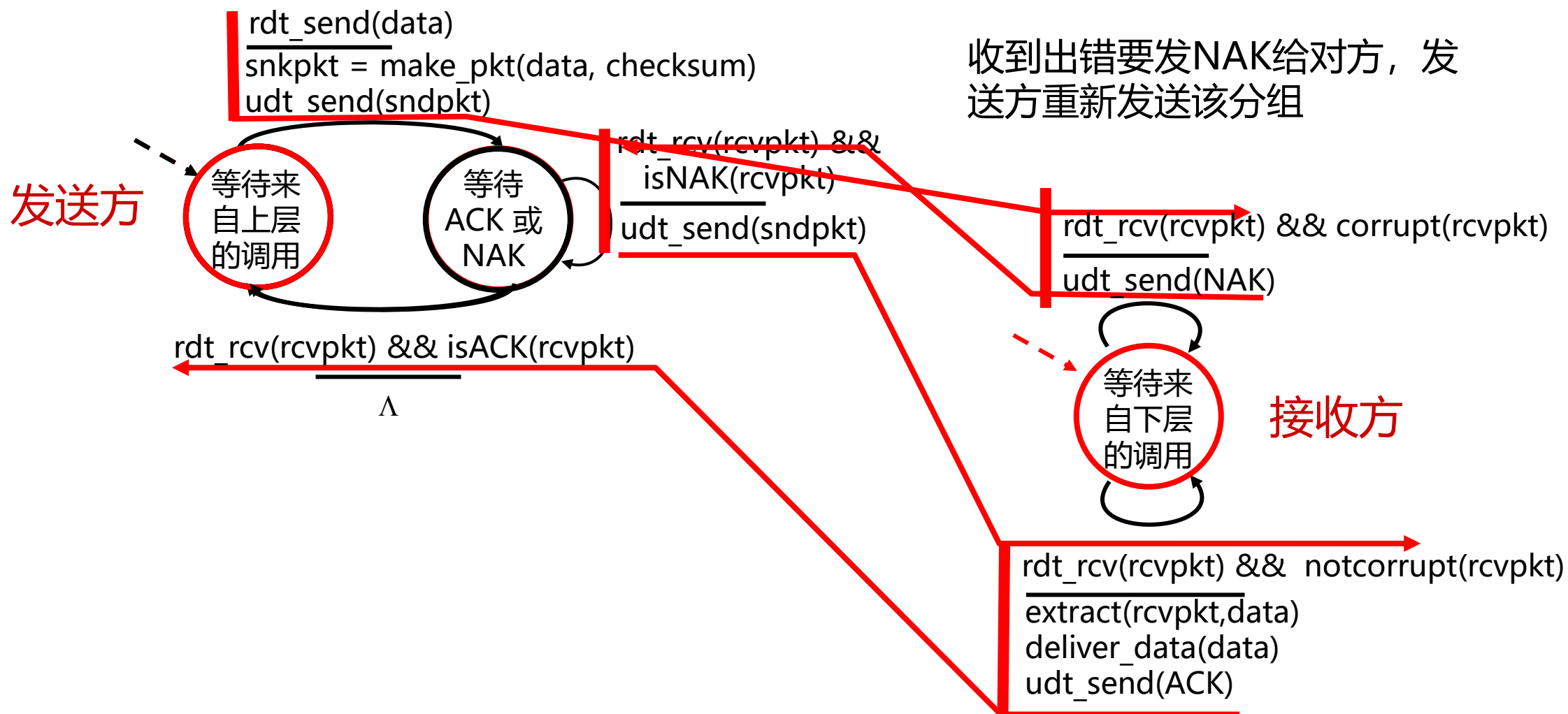
注意：从发送方的角度，并不知道接收方的状态。一定要依赖于特定的消息才能进行下一步的反应

- 这就是为什么需要预先定义协议！

rdt2.0: 无差错时的操作



rdt2.0: 出现差错场景



rdt2.0 有致命错误! -> rdt 2.1

如果 ACK/NAK 出错怎么办?

- 发送方不知道接收方发生了什么事情!
- 发送方如何做?
 - 重传? 可能重复
 - 不重传? 可能死锁/出错
- 需要引入新机制
 - 序号 (sequence number)

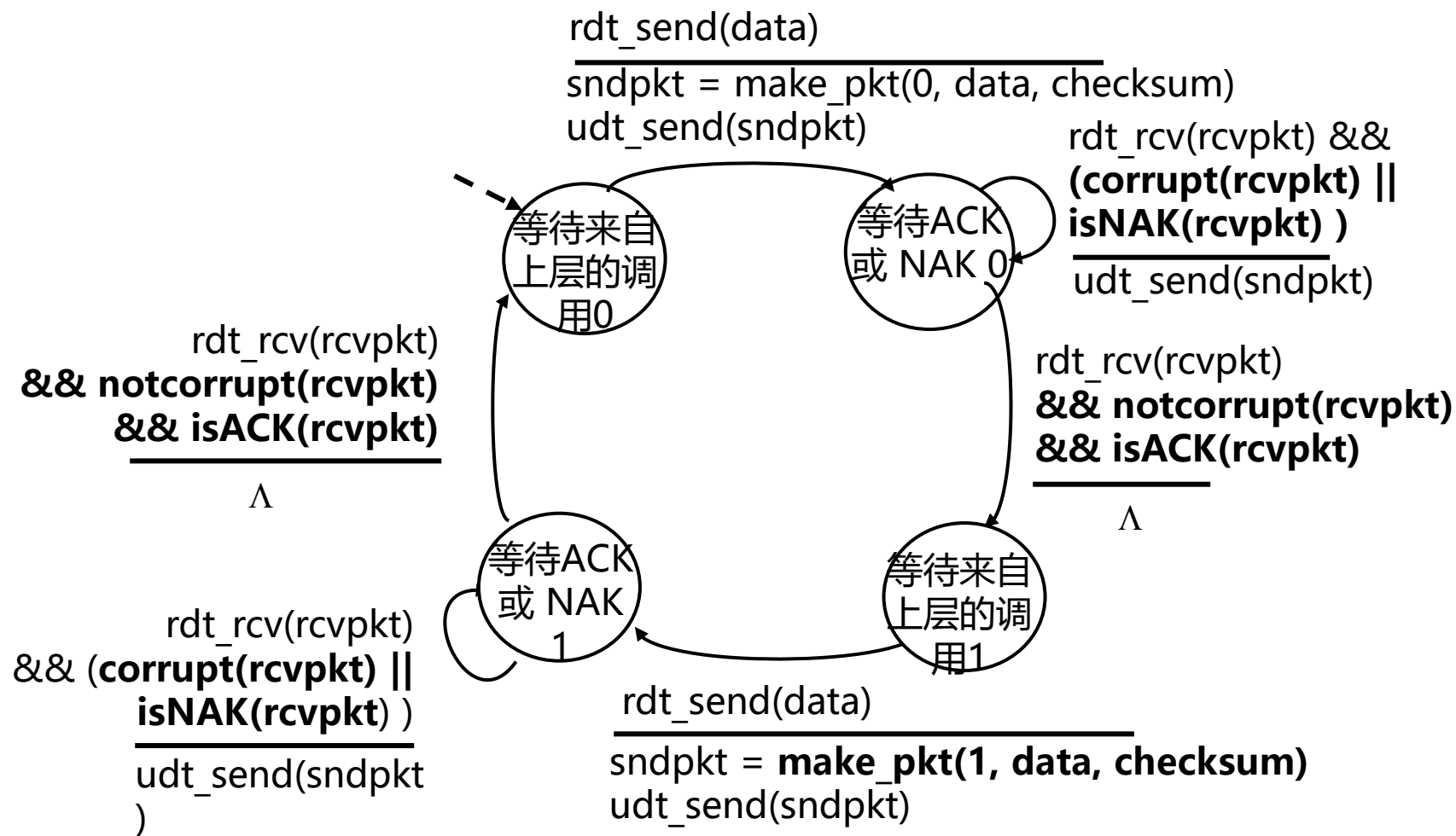
处理重复:

- 发送方在每个分组中加入序号
- 如果ACK/NAK出错, 发送方重传当前分组
- 接收方可以丢弃重复分组
 - 冗余分组 (duplicate packet)

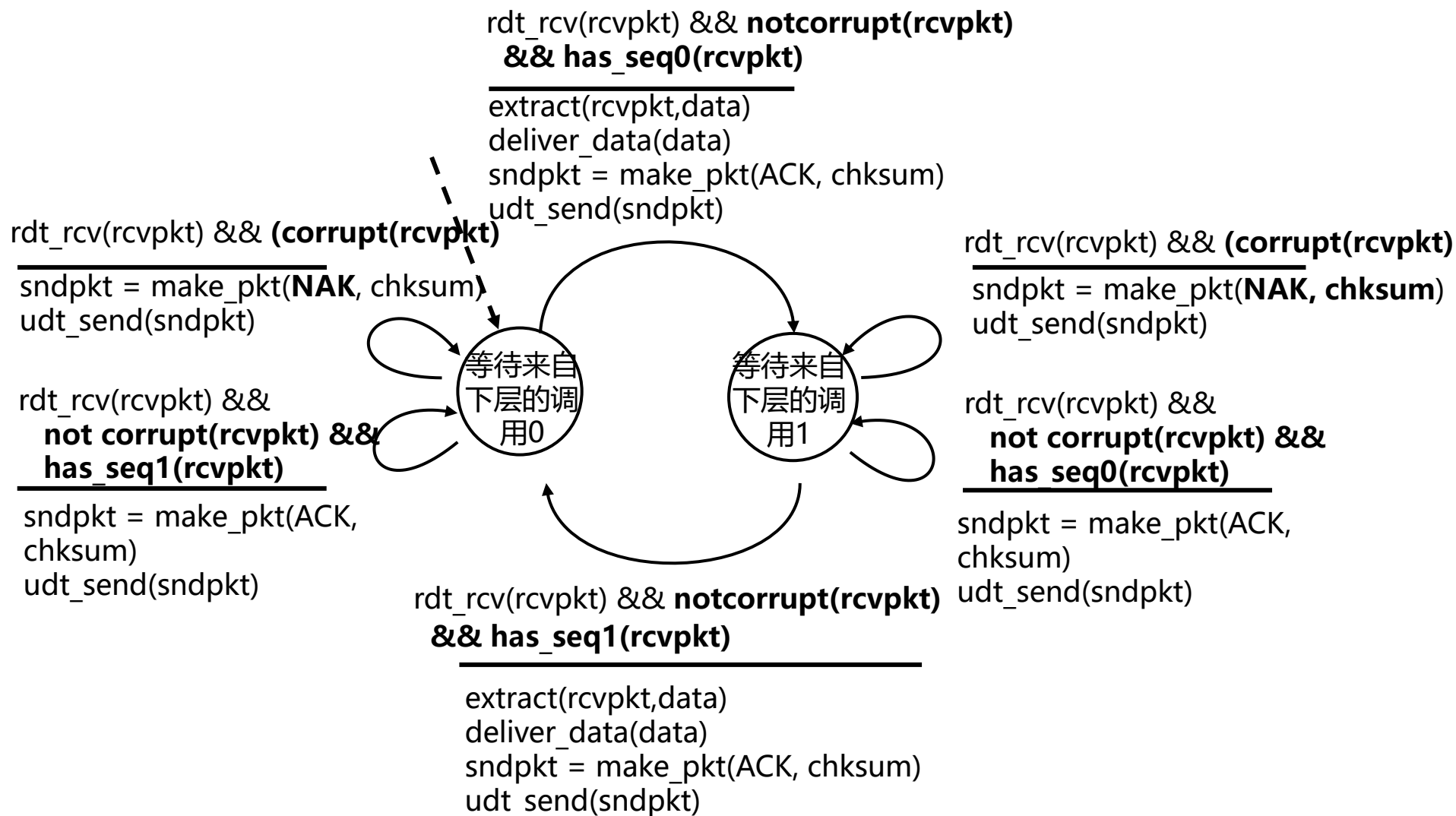
停等协议

发送方每发送一个分组,
都要等待接收方回复

rdt2.1: 发送方处理出错的ACK/NAK



rdt2.1: 接收方处理出错的ACK/NAK



rdt2.1: 讨论

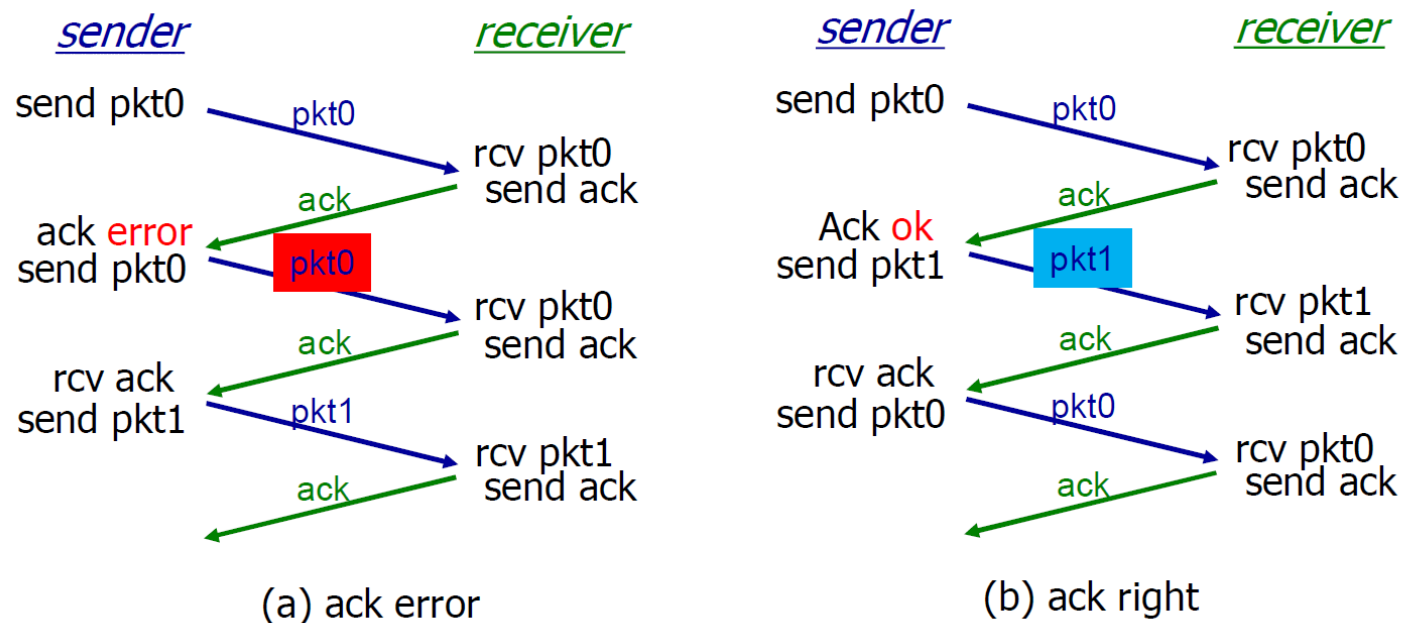
发送方:

- 在分组中加入序列号
- 两个序列号 (0, 1) 就足够了
 - 一次只发送一个未经确认的分组
- 必须检测ACK/NAK是否出错 (需要EDC)
- 状态数变成了两倍
 - 必须记住当前分组的序列号为0还是1

接收方:

- 必须检测接收到的分组是否是重复的
 - 状态会指示希望接收到的分组的序号为0还是1
- 注意: 接收方并不知道发送方是否正确收到了其ACK/NAK
 - 没有安排确认的确认
 - 具体解释见下页

rdt2.1: 示例



- 接收方不知道它最后发送的ACK/NAK是否被正确地收到
 - 发送方不对收到的ack/nak给确认，没有所谓的确认的确认；
 - 接收方发送ack，如果后面接收方收到的是：
 - 分组pkt0？ 则说明之前ack 错误，对方没收到，可以继续重发ack
 - 下一个分组？ pkt1， ack正确

rdt2.2: 无NAK的协议

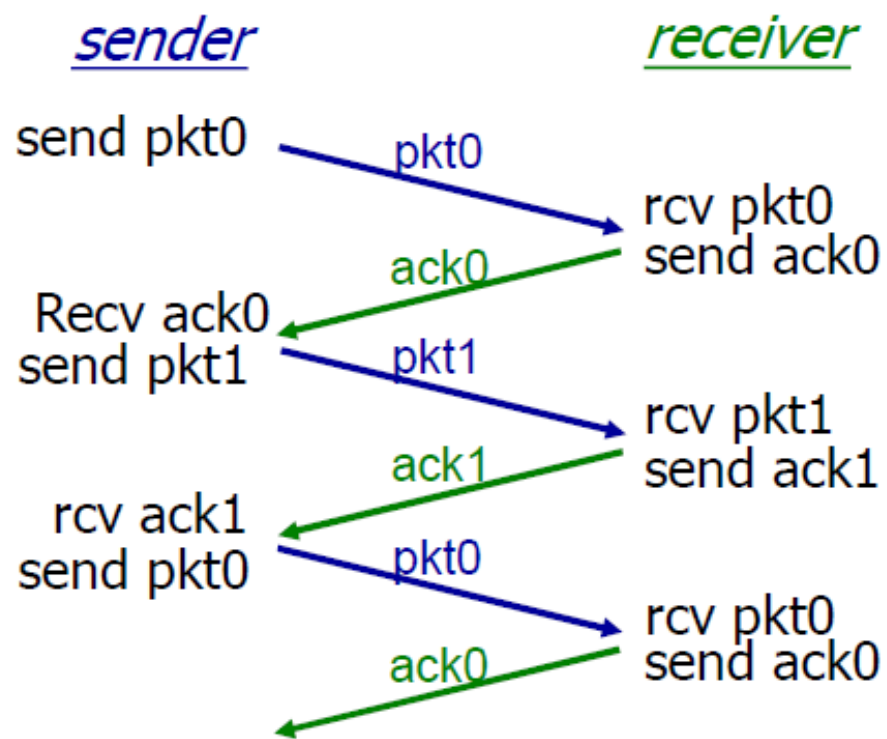
- 功能同rdt2.1，但只使用ACK (ack 要编号)
- 接收方对最后正确接收的分组发ACK，以替代NAK
 - 接收方必须显式地包含被正确接收分组的序号
- 当收到重复的ACK (如：再次收到ack0) 时，发送方与收到NAK采取相同的动作：重传当前分组
- 为后面的一次发送多个数据单位做一个准备
 - 一次能够发送多个
 - 每一个的应答都有：ACK, NAK; 麻烦
 - 使用对前一个数据单位的ACK，代替本数据单位的NAK
 - 确认信息减少一半，协议处理简单

NAK free

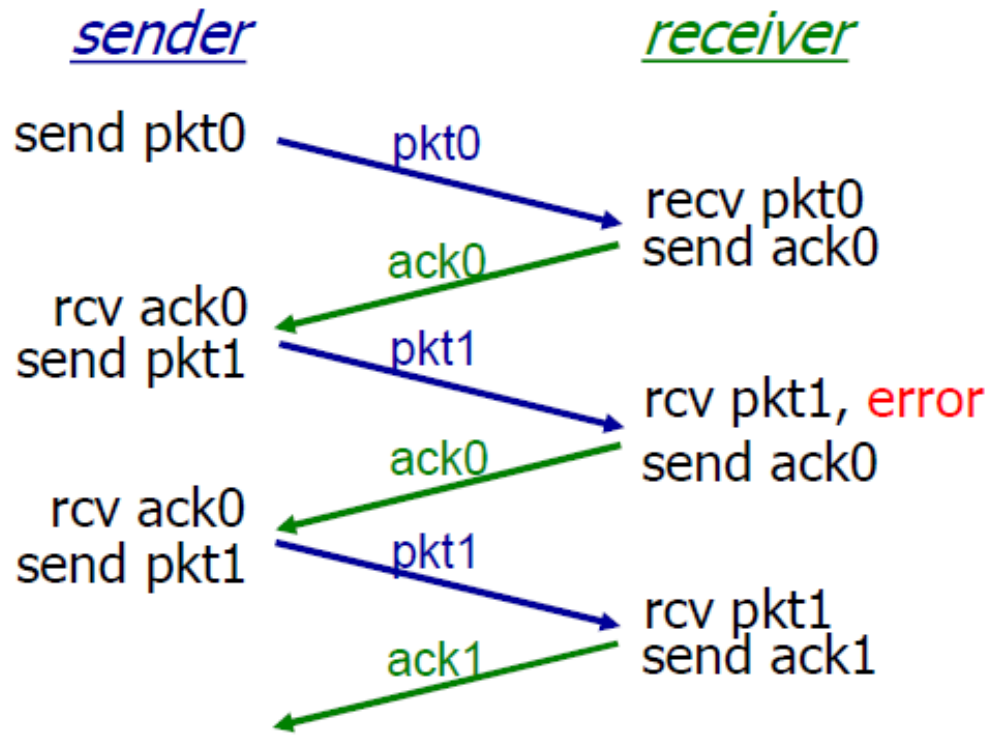


- 每一个的应答都有：ACK, NAK; 麻烦
- 使用对前一个数据单位的ACK, 代替本数据单位的NAK
- 确认信息减少一半, 协议处理简单

rdt 2.2 的运行

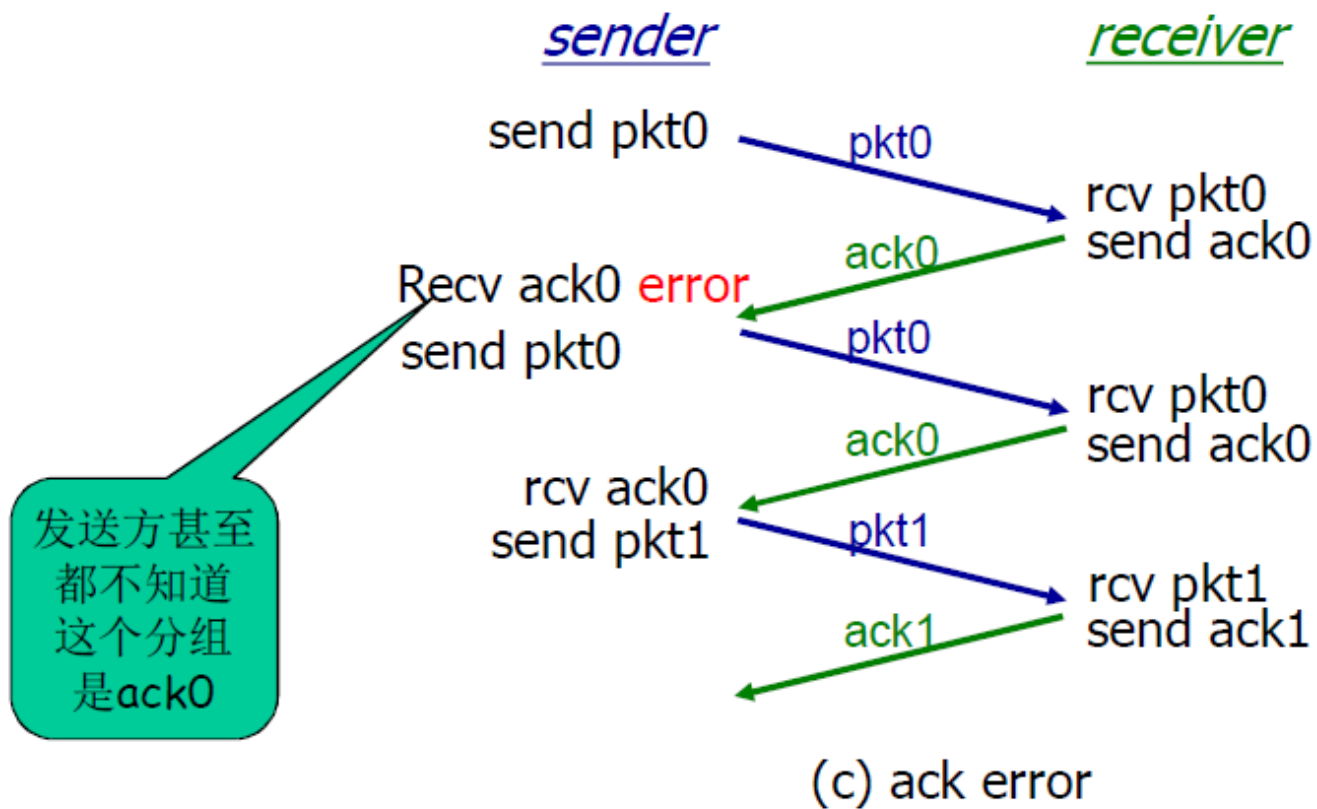


(a) No error (packet or ack error)

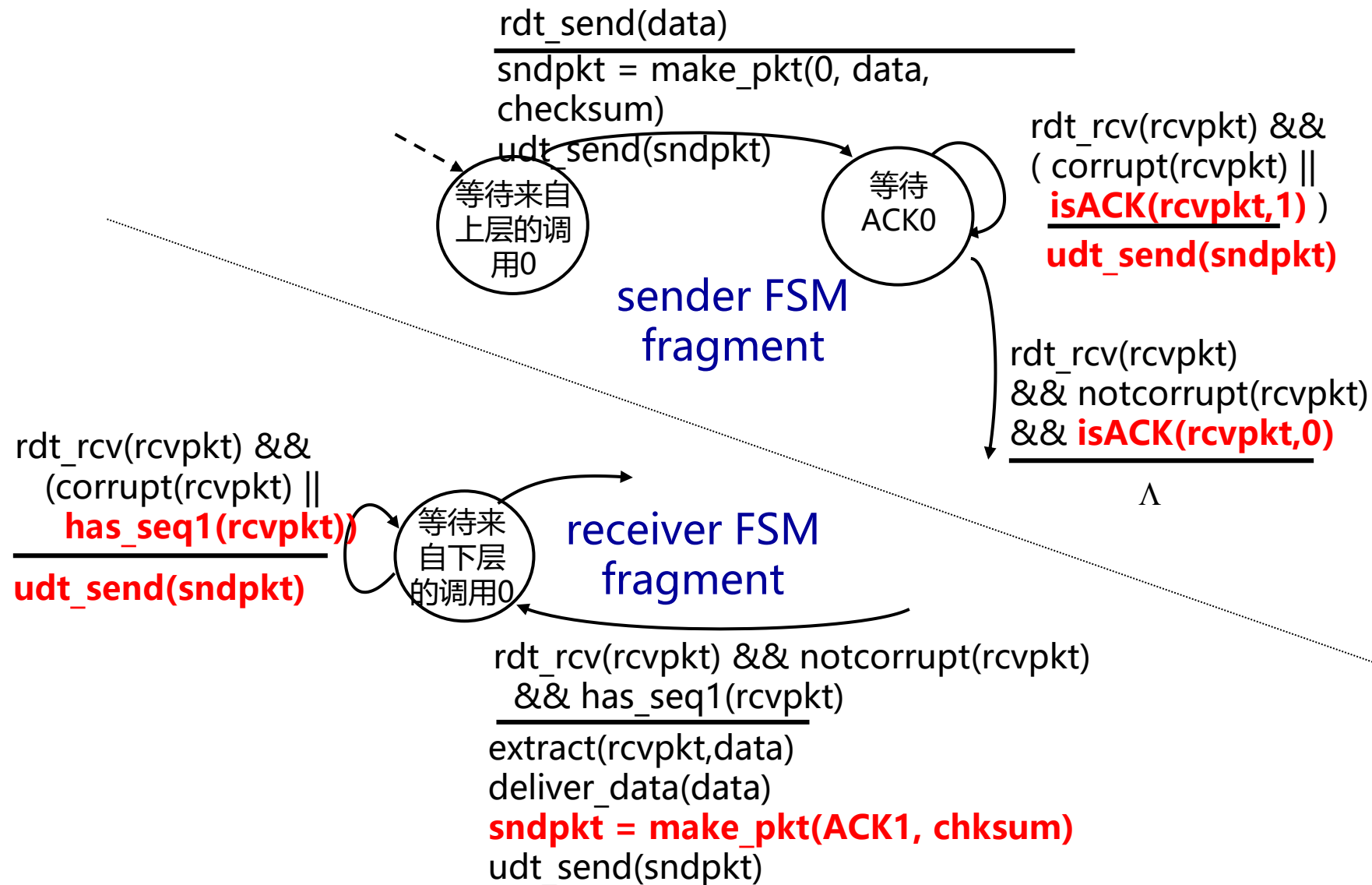


(b) packet error

rdt 2.2 的运行



rdt2.2: 收、发方片段



rdt3.0: 具有比特差错和分组丢失的信道

- **新的假设:** 下层信道可能会丢失分组 (数据或ACK)
 - 会死锁
 - 前述机制无法处理这种状况:
 - 检验和
 - 序列号
 - ACK
 - 重传
- **方法:** 发送方等待ACK一段**合理的时间**
 - 发送端超时重传: 如果到时没有收到ACK->重传
 - 问题: 如果分组 (或ACK) 只是被延迟了:
 - 重传将会导致数据重复, 但利用**序列号**已经可以处理这个问题
 - 接收方必须指明被正确接收的序列号
 - 需要一个倒数计时器

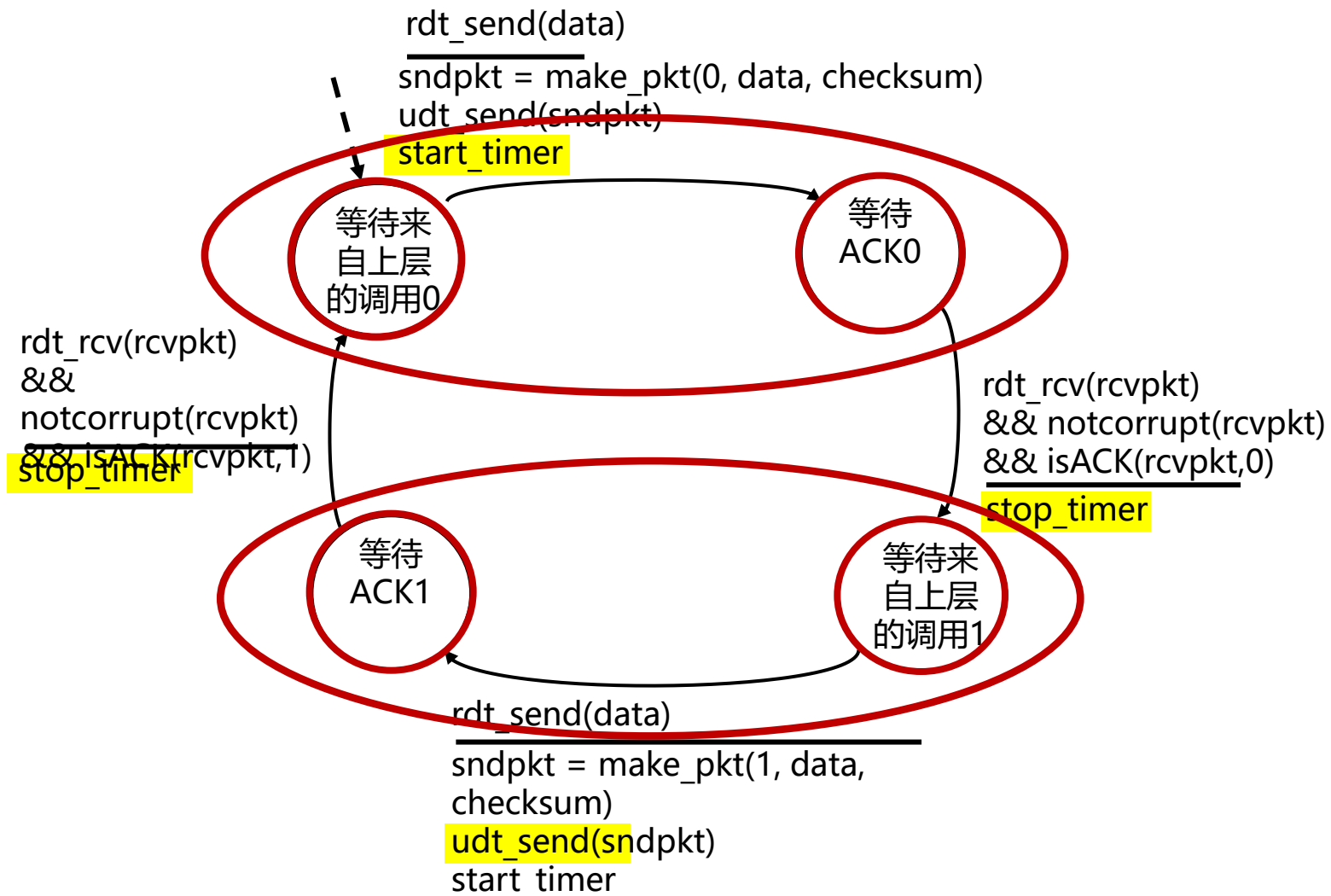
问题

1. 怎么检测丢包?
2. 发生丢包后该做什么?

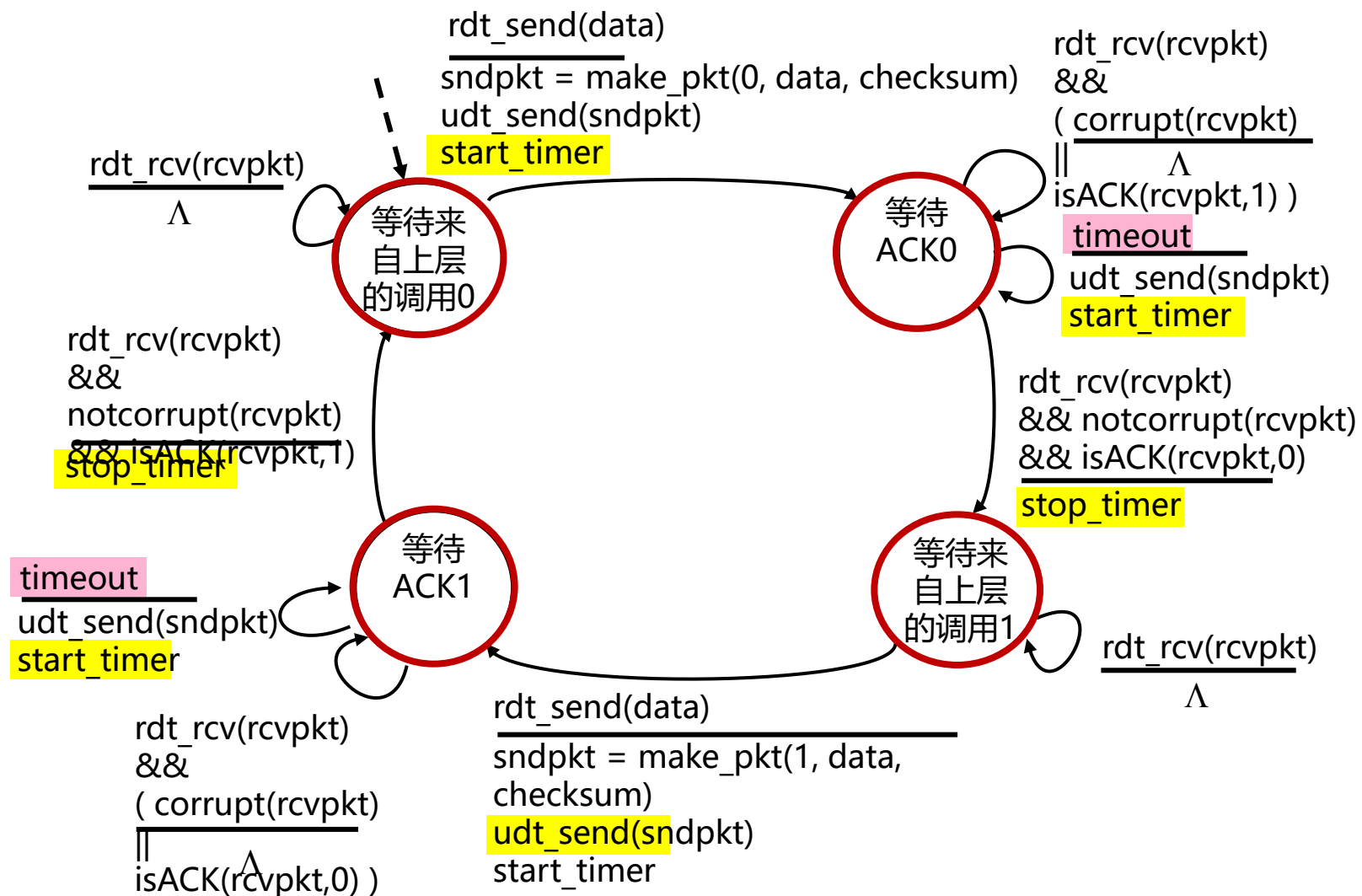


timeout

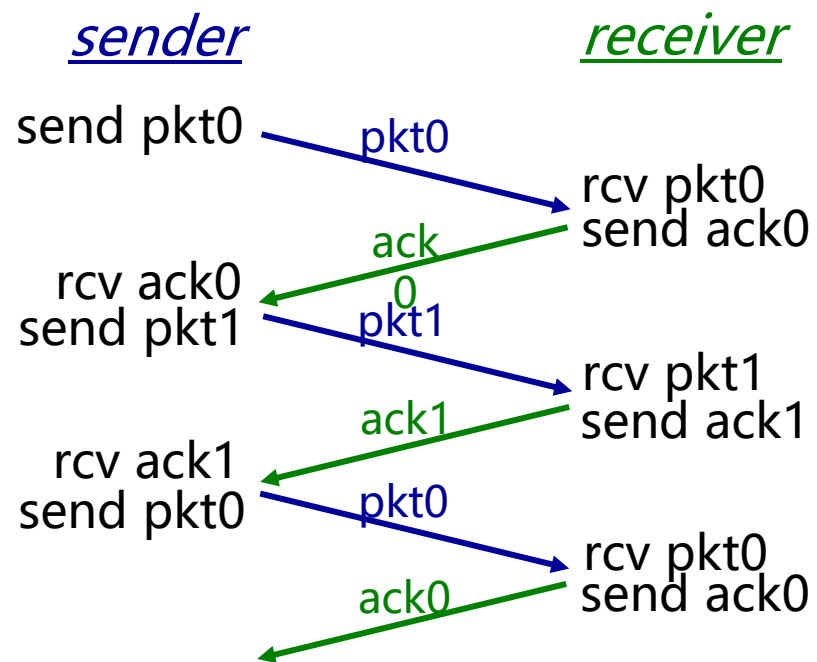
rdt3.0 发送方



rdt3.0 发送方

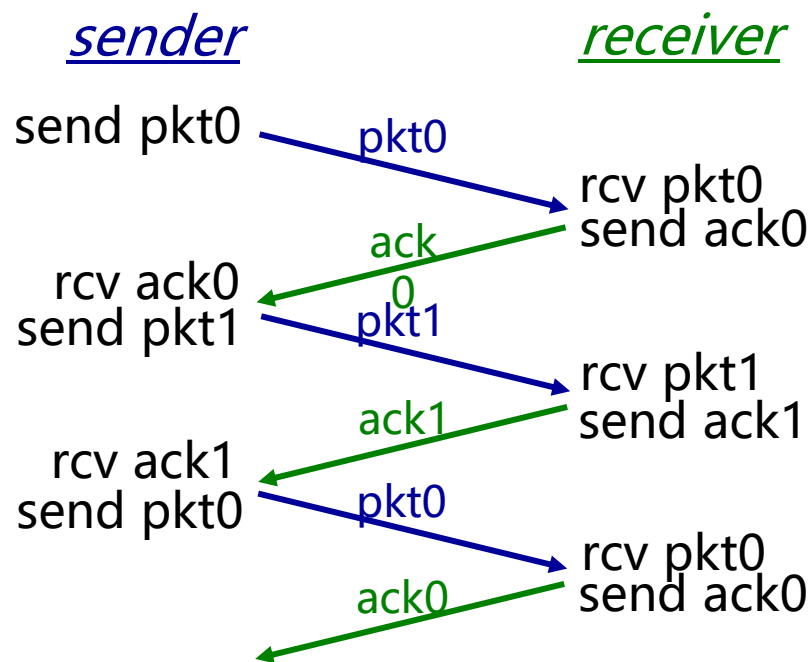


rdt3.0 的运行

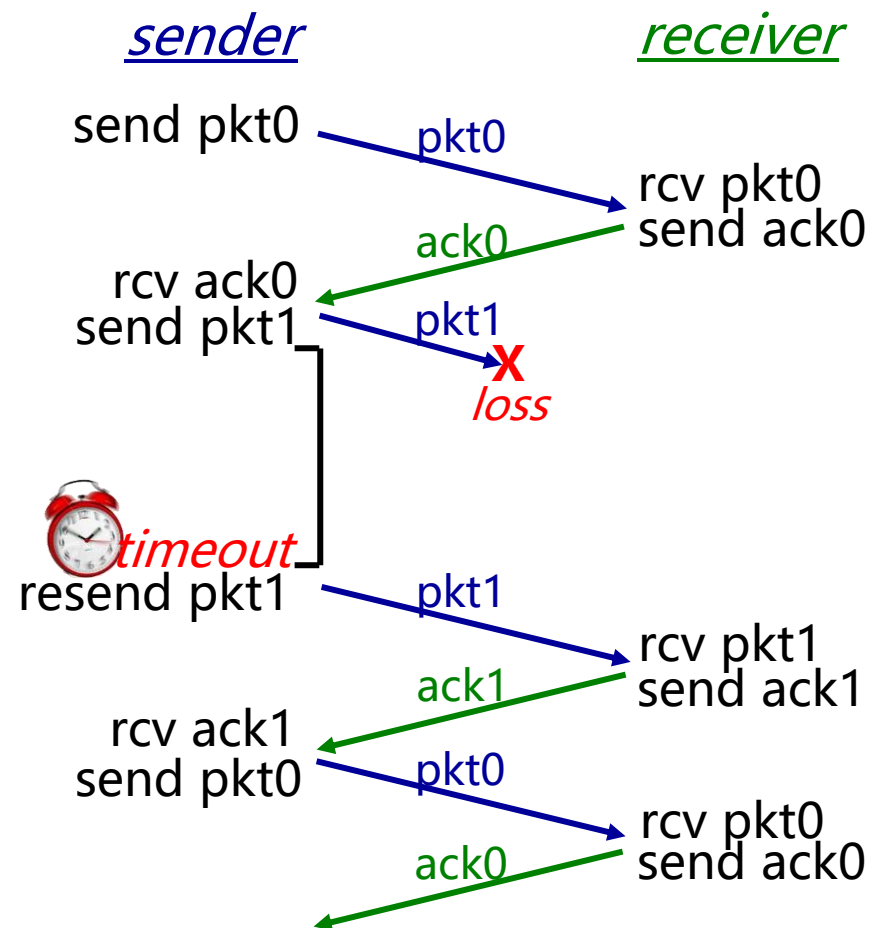


(a) no loss

rdt3.0 的运行

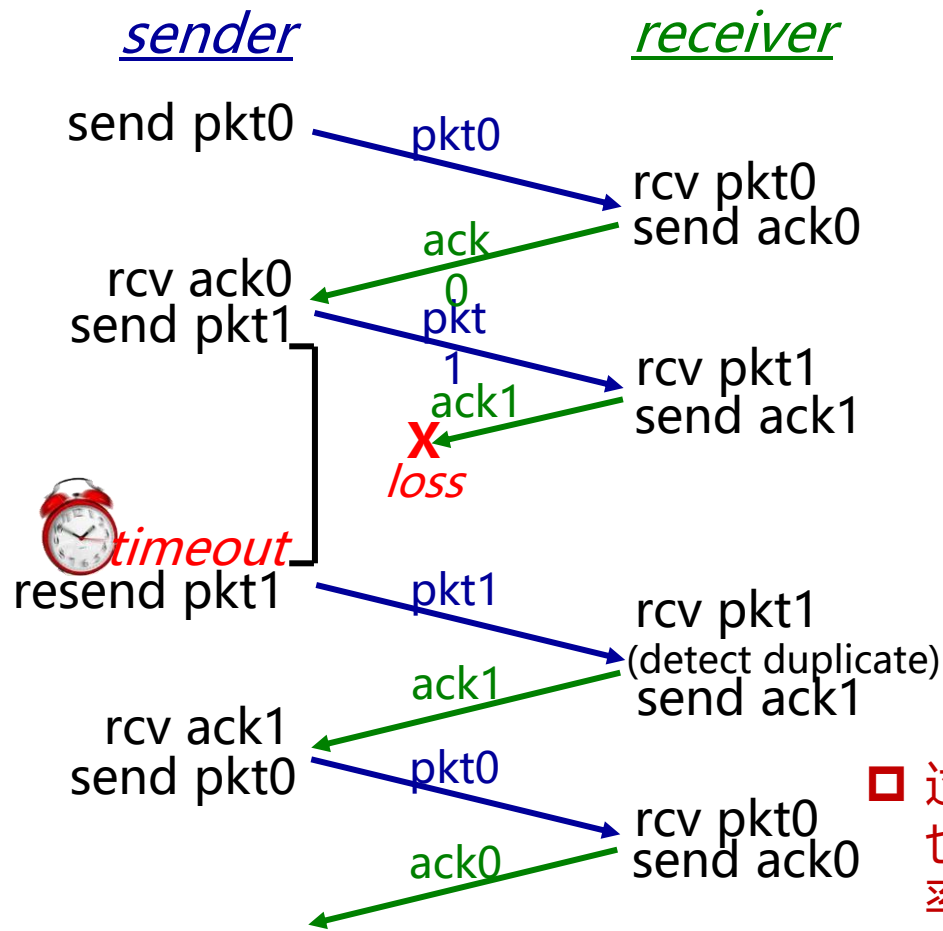


(a) no loss



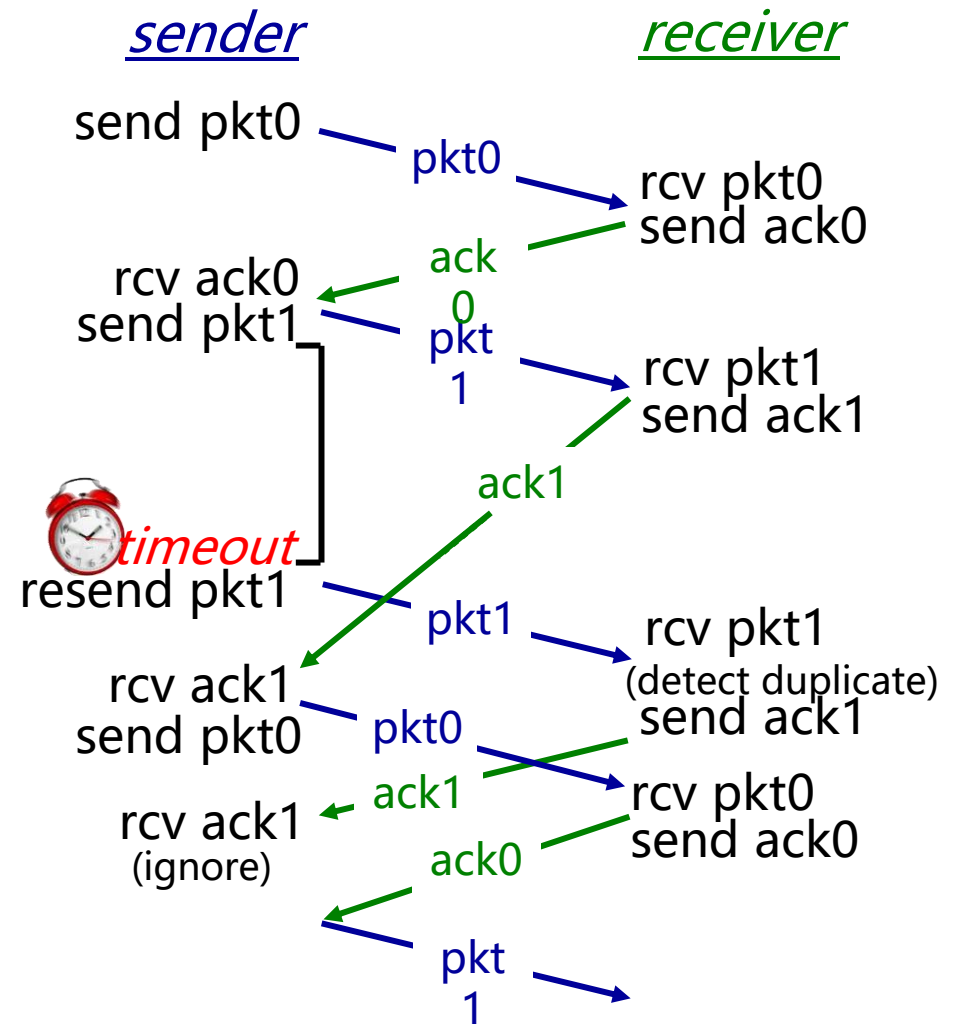
(b) packet loss

rdt3.0 的运行



(c) ACK loss

- 过早超时 (延迟的ACK) 也能够正常工作; 但效率较低, 一半的分组和确认是重复的;
- 设置一个合理的超时时间也是比较重要的。



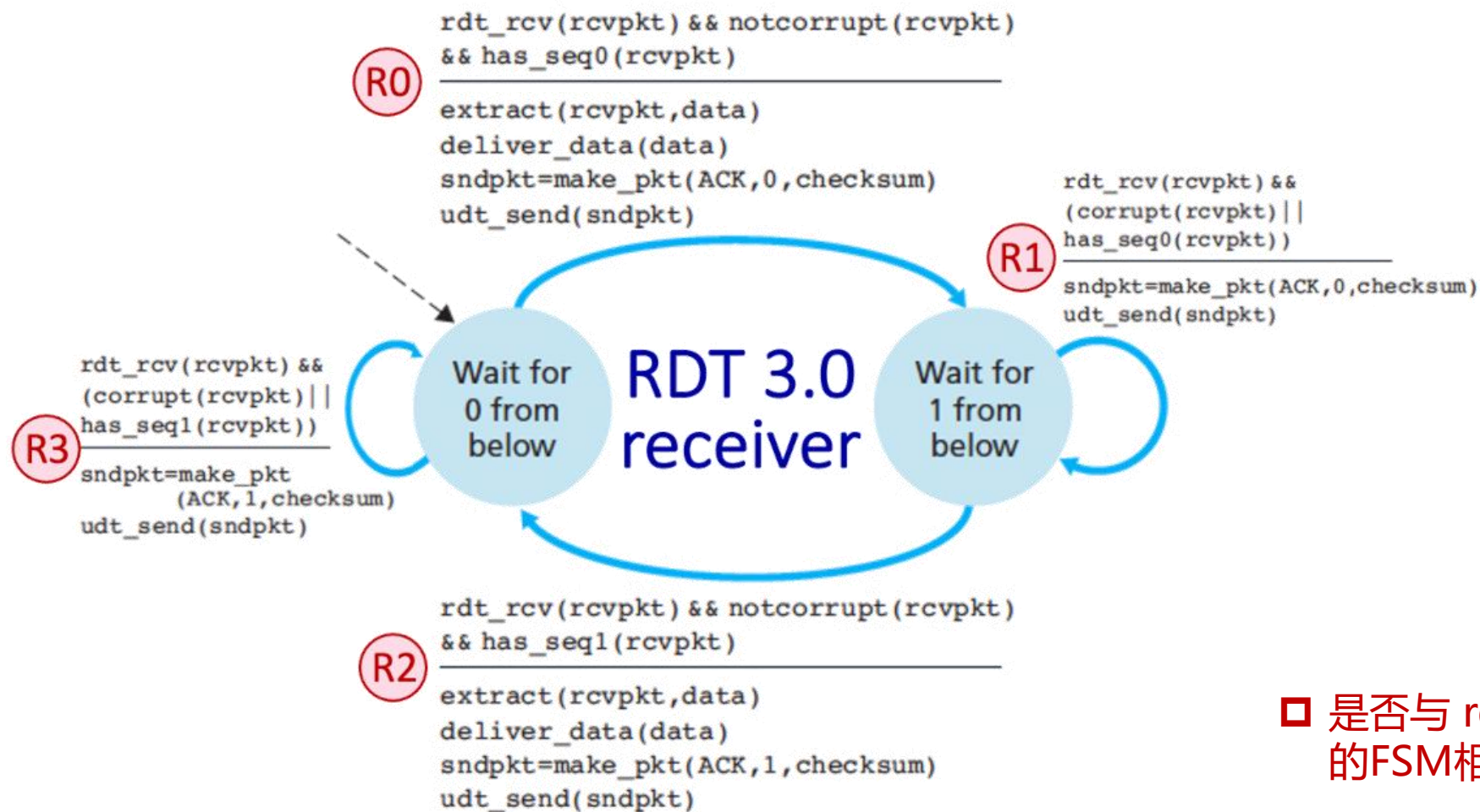
(d) premature timeout/ delayed ACK

rdt3.0 接收方

- 接收方的FSM?

- 2个状态
- 等待来自下层的0
 - 接收分组0, 检测无错误, 发送ACK0
 - 接收分组0, 检测错误, 发送ACK1
 - 接收分组1 (检测冗余), 发送ACK1
- 等待来自下层的1
 - 接收分组1, 检测无错误, 发送ACK1
 - 接收分组1, 检测错误, 发送ACK0
 - 接收分组0 (检测冗余), 发送ACK0

rdt3.0 接收方



□ 是否与 rdt 2.2 接收方的FSM相同?

rdt2.2 接收方 (补充)

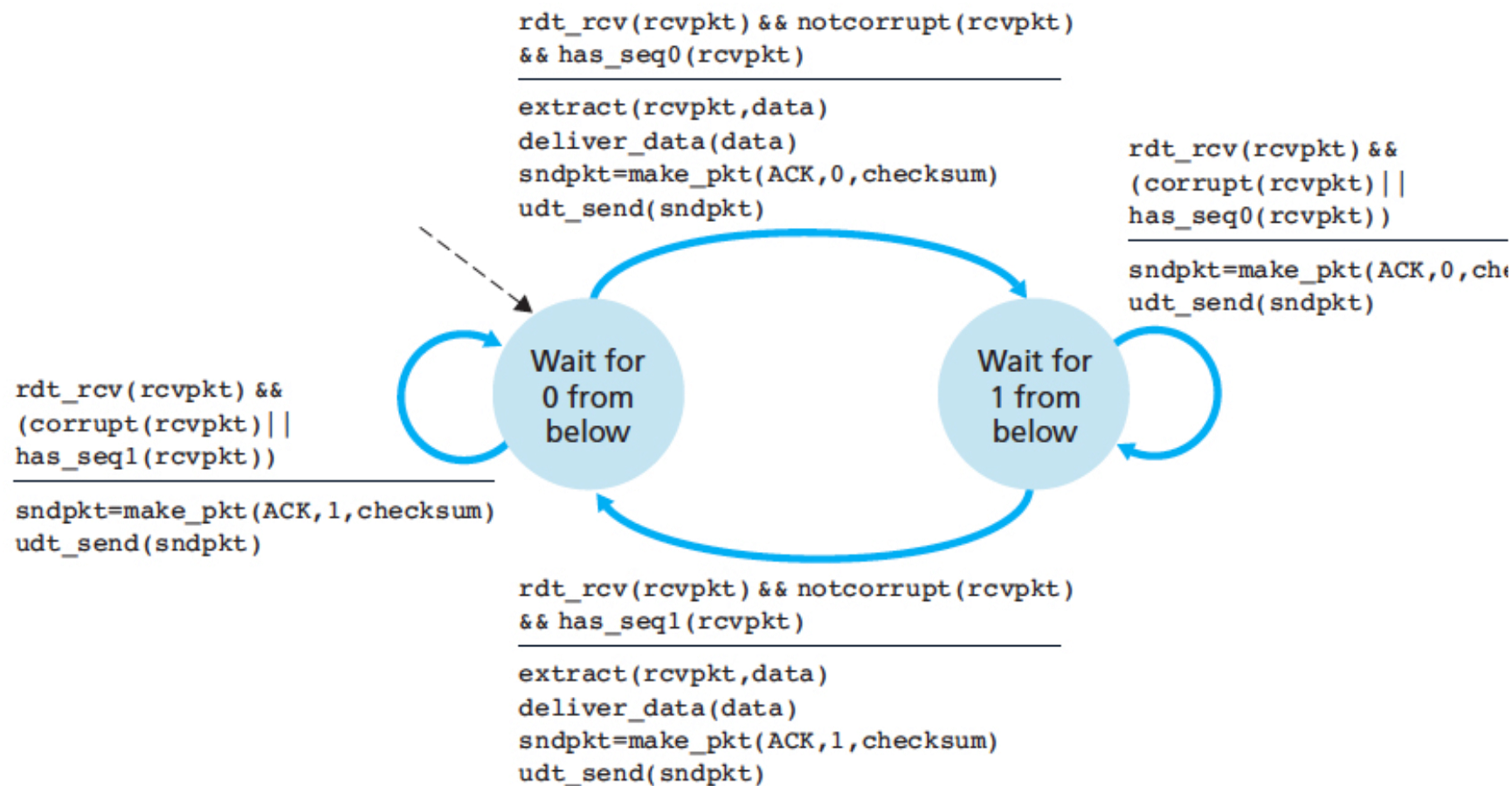


Figure 3.14 ♦ rdt2.2 receiver

rdt2.2 发送方 (补充)

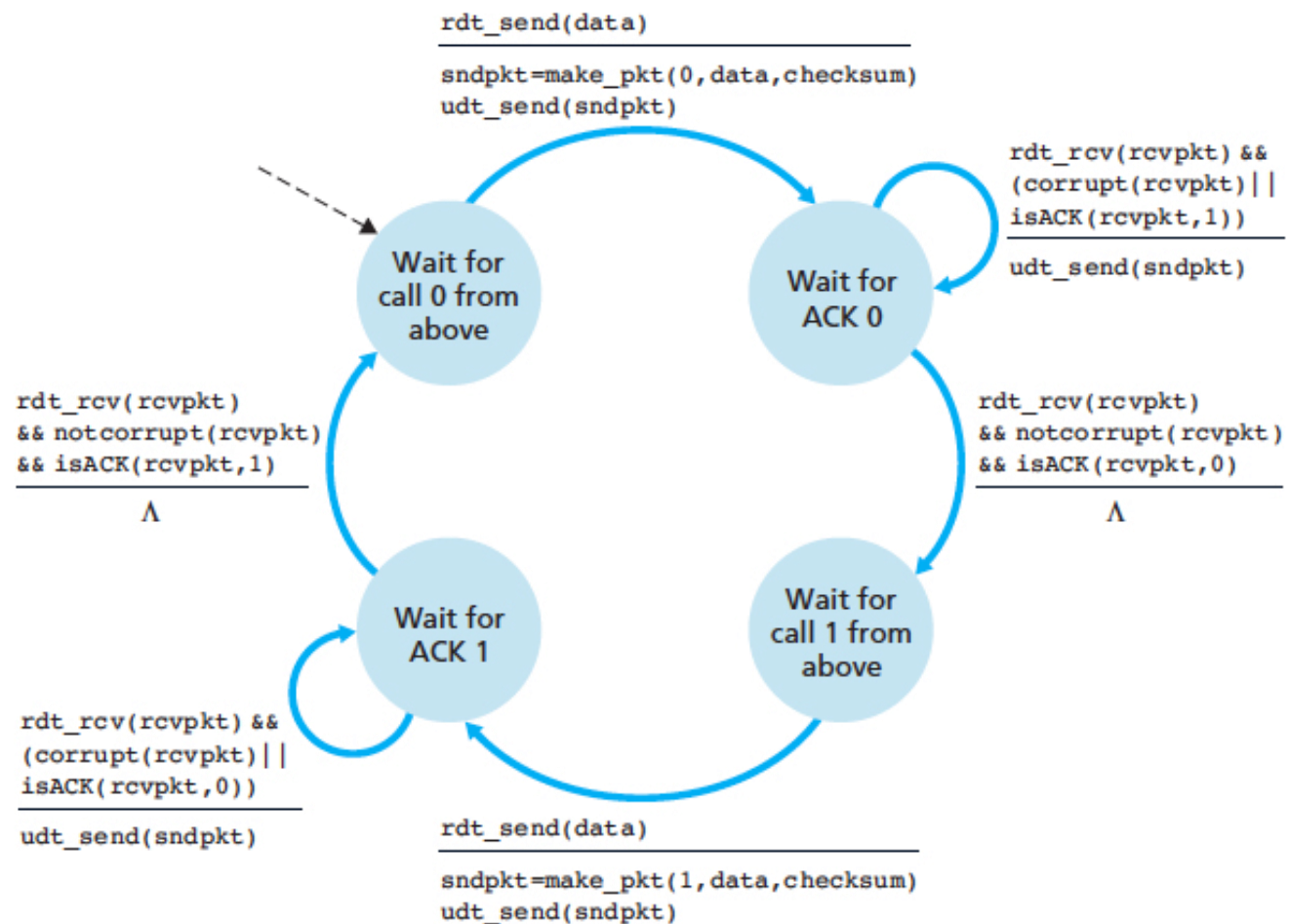


Figure 3.13 ♦ rdt2.2 sender

小结/回顾: rdt 1.0 – 3.0

- 假设: 信道是可靠的
 - 1.0 – 可靠传输
- 假设: 信道存在比特受损 (差错)
 - 2.0 – ACK/NAK (去掉可靠假设, 传输会出错)
 - 2.1 – 处理ACK/NAK错误, 对发送分组标号 (等0, 等1)
 - 2.2 – NAK FREE (简化表述 $\text{NAK}_N = \text{ACK}_{(n-1)}$)
- 假设: 比特差错+丢包
 - 3.0 – Timer (增加定时器, 应对报文丢失情况)